ADR 1: Separate business rules from the UI layer

Context

CampusPlan uses a console interface to manage students, availability and draft events. FR-01 and FR-02 require consistent validation, and NFR-04 requires application operations to be testable without the UI. Keeping business rules in ConsoleUI would make those operations depend on console interaction and make future interface changes harder.

Alternatives considered

1. Validate business rules inside ConsoleUI. This is quick to start, but mixes interaction with application behaviour and makes isolated tests harder.

2. Put all validation in domain objects. This suits rules about one object, but cross-record checks such as unique emails and valid invitee IDs need access to other records.

3. Separate the UI, application services and domain objects. This supports direct testing and gives each component a clear responsibility. This is the proposed choice.

Decision

ConsoleUI will collect input, parse values, call application services and display results. StudentService, RoomCatalogService, AvailabilityService and EventPlanningService will coordinate their respective operations. EventDraftRequest will carry event-planning input to EventPlanningService.

Domain classes such as Student, Event and TimeInterval will enforce rules local to their own values. Services will enforce cross-record rules using CampusPlanData, including duplicate emails and references to existing students. Business validation will run before state changes. ConsoleUI will display ValidationException messages without duplicating the business rules. StorageService will coordinate persistence separately.

Consequences

Benefits: Application operations can be tested directly without console input. Team members can work against agreed method contracts, and UI changes are less likely to affect business rules.

Costs: More classes and contracts must be maintained. The team must agree which rules belong in domain objects and which require a service to avoid duplicated checks.

Validation evidence

Planned checks: call the services directly in JUnit 5; test valid student creation, duplicate normalized emails, invalid intervals and invalid draft-event constraints. Verify rejected mutations

preserve existing records. Check that ConsoleUI delegates operations to these services. Add actual test names and results after execution.

ADR 2: Use JSON storage through a DataStore interface

Context

FR-03 requires students, rooms, availability and draft events to survive a restart with their IDs preserved. The brief requires UTF-8 JSON, an empty start when storage is missing, and a clear error for corrupt storage. NFR-05 also requires a failed save to preserve the previous valid file. Application tests need an option that does not depend on real files.

Alternatives considered

1. Read and write JSON inside ConsoleUI or individual services. This needs less initial structure, but spreads file handling across the application and makes failures harder to test consistently.

2. Use one concrete JSON storage class everywhere. This centralizes file access, but application tests still depend on the filesystem.

3. Use DataStore with JSON and in-memory implementations. This separates storage from application operations and supports isolated tests. This is the proposed choice.

Decision

StorageService will use the DataStore interface. JsonDataStore will load and save CampusPlanData as UTF-8 JSON. InMemoryDataStore will support application tests without filesystem access. The implementations will follow the same contract and will not expose mutable internal state unintentionally.

Loading will validate the complete dataset before replacing current application data, preserving saved IDs and checking references. A missing file will start an empty application. SeedDataLoader will provide an explicit seed-data load only when the application is empty. Invalid or corrupt storage will produce a clear error and must not be automatically overwritten.

Saving will write a complete temporary file in the same directory before replacing the valid file using an atomic move where supported. If safe replacement cannot be completed, the save will fail clearly and preserve the previous valid file. Storage failures will be reported through StorageException and displayed by the UI.

Consequences

Benefits: Application tests can use in-memory storage, while integration tests exercise actual JSON files. Centralized failure handling reduces the chance of silently losing saved records.

Costs: Two implementations and data-format validation must be maintained. Temporary-file handling and platform differences require explicit failure tests. The in-memory implementation cannot prove file-storage correctness.

Validation evidence

Planned checks: save and reload a complete P1 dataset in a temporary directory and compare records, relationships and IDs. Test missing storage, malformed JSON, invalid references, seed loading into a nonempty application and simulated save failures. Verify failed loading preserves current state and failed saving preserves the previous valid file. Add actual test names and results after execution.

CampusPlan — Interface Contracts

Interface contracts

For event planning, storage, scheduling and undo/redo: inputs, results, validation failures, and who owns mutable data. Signatures match the team's P1 class diagram; behaviour follows the project brief and UC4–UC6 in requirements.pdf. Event planning and storage are built in P1; scheduling and undo/redo are planned for P2.

1. Rules that apply to every service

- **Validate before mutating.** On any failure a service throws a subclass of CampusPlanException and leaves all data unchanged. There are no partial changes (NFR-02).
- **One shared data object.** Services receive the same CampusPlanData and change only that. They never read the console or touch files.
- **ConsoleUI holds no rules.** It calls a service, and catches CampusPlanException to print its message (ADR-01).
- **Exception meanings.** ValidationException: an input or the current state breaks a rule. NotFoundException: a referenced id does not exist. DuplicateEmailException: a student email is already used. StorageException: a file could not be read or written. CorruptDataException: a saved file exists but is invalid.

2. Event planning — EventPlanningService (P1)

createDraft(req: EventDraftRequest): Event

- **Input:** EventDraftRequest (title, inviteeIds, windowStart, windowEnd, durationMinutes, requiredFeatures, minAttendance).
- **Returns:** a new Event with a freshly generated id, status = DRAFT, responses empty, roomId/slot unset.
- **Throws ValidationException** when: title is empty after trimming or exceeds 100 characters (E1); fewer than 2 or more than 20 students invited (E2); the invitee list contains a duplicate student (E3); the date window is less than 1 day or more than 7 consecutive days (E4); duration is less than 30 min, more than 180 min, or not a 30-minute increment (E5); minimum attendance is less than 2 or greater than the number of invited students (E6).
- **Throws NotFoundException** when one or more invited students don't exist (E7).
- **Ownership:** caller passes an immutable EventDraftRequest; the service owns creation of the new Event and its id (via idx: IdGenerator). No existing data is mutated.

editDraft(id: String, req: EventDraftRequest): Event

- **Input:** an existing event id, plus a replacement EventDraftRequest.
- **Returns:** the updated Event with the same id, fields replaced from req.
- **Throws NotFoundException** when id doesn't match any event.
- **Throws ValidationException** when: the event's status is not DRAFT (scheduled/cancelled events can't be edited — cancel and create a new one instead); or any of the same field-level checks as createDraft (E1–E7) fail on the new values.
- **Ownership:** mutates the existing Event identified by id in place (via CampusPlanData). No other event is touched.

get(id: String): Event

- **Input:** an event id.
- **Returns:** the matching Event.
- **Throws NotFoundException** when id doesn't match any event.
- **Ownership:** read-only, no mutation.

listAll(): List<Event>

- **Input:** none.
- **Returns:** all events currently held in CampusPlanData, in no specified order (caller sorts/filters as needed).
- **Throws:** nothing — an empty list is a valid result, not an error.
- **Ownership:** read-only, no mutation.

3. Storage — StorageService and DataStore (P1)

StorageService.startUp(): CampusPlanData

- **Input:** none (uses the DataStore it was built with).
- **Returns:** the single live CampusPlanData that all services share. If a saved file exists it is loaded with IDs and responses preserved. If no file exists, an empty CampusPlanData is returned and the organizer is offered the seed load (UC5 A1, A2).
- **Throws CorruptDataException** when the saved file exists but is invalid (UC5 E1); the file is not overwritten. **Throws StorageException** when the file cannot be read.
- **Ownership:** StorageService creates the CampusPlanData and hands the same instance to every service. No other class creates one.

StorageService.save(): void

- **Input:** none. Saves the current CampusPlanData; the organizer triggers saves manually.
- **Returns:** nothing.
- **Throws StorageException** when the write fails (UC5 E2). The previous valid file is kept, because JsonDataStore writes a temporary file and renames it only after the write succeeds (NFR-05).
- **Ownership:** read-only on CampusPlanData.

StorageService.loadSeed(seedFile: Path): void

- **Input:** the path of the seed JSON file.
- **Returns:** nothing. Fills the empty CampusPlanData with the sample students, rooms, availability and events.
- **Throws ValidationException** when CampusPlanData is not empty (UC5 E3). **Throws CorruptDataException** when the seed file is invalid, and **StorageException** when it cannot be read.
- **Ownership:** changes CampusPlanData only while it is empty. Seed loading is outside undo/redo.

StorageService.getData(): CampusPlanData

- **Input:** none. **Returns:** the live shared instance. **Throws:** nothing.
- **Ownership:** callers must not replace it. Only the services change it, through their own methods.

DataStore (interface): exists, load, save

- **exists(): boolean** — true when saved data is present. A missing file is not an error.
- **load(): CampusPlanData** — returns the saved data. Throws CorruptDataException for invalid content and StorageException for an I/O failure.
- **save(data: CampusPlanData): void** — stores the data. Throws StorageException, and must never leave a half-written file in place of the last valid one.
- **Implementations:** JsonDataStore (UTF-8 JSON file, written atomically) and InMemoryDataStore (for tests, holds at most one saved CampusPlanData). Both follow the same three rules, so a test written against one also passes against the other.

4. Scheduling — SchedulingService (planned P2)

Signatures are from the team diagram. Behaviour follows section 3 of the brief (scheduling rules), FR-04 and FR-05.

suggest(eventId: String): SuggestionResult

- **Input:** the id of a DRAFT event.
- **Returns:** a SuggestionResult with up to 5 Suggestions plus rejectedForRoom and rejectedForAttendance counts. Each Suggestion has roomId, slot, eligibleIds and ineligibleReasons (student id to reason). Ranking: eligible count descending, then date/time ascending, then unused room seats ascending, then room ID ascending. Room failures are counted first so the two rejected counts never overlap. Repeating the call returns the same order.
- **Throws NotFoundException** when the event does not exist (UC6 E1). **Throws ValidationException** when the event is not DRAFT.
- **Ownership:** read-only. Selecting a suggestion does not reserve it.

confirm(eventId: String, s: Suggestion): Event

- **Input:** the event id and the chosen Suggestion.
- **Returns:** the updated Event with status SCHEDULED and roomId and slot set.
- **Throws NotFoundException** when the event or room does not exist. **Throws ValidationException** when revalidation fails (the suggestion is stale, the room is taken by another SCHEDULED event, capacity or features no longer fit, or eligible attendance is below the minimum) or when the event is not DRAFT.
- **Ownership:** changes the Event and the room reservation together. It is all or nothing, so a rejection changes nothing (FR-05, NFR-02).

cancel(eventId: String): void

- **Input:** the id of a SCHEDULED event. The UI asks for confirmation first; declining never calls this method.
- **Returns:** nothing. Status becomes CANCELLED, and the room and participant commitments are freed. The record and responses are kept. A cancelled event cannot be scheduled again.
- **Throws NotFoundException** when the event does not exist. **Throws ValidationException** when the event is not SCHEDULED.
- **Ownership:** changes the Event and releases its reservation together.

5. Undo/redo — Command and CommandHistory (planned P2)

This is the Command pattern: each user mutation is wrapped in a Command that can undo itself.

Command (interface)

- **execute(): void** — applies one user mutation. Throws the same exceptions as the service it wraps, with no partial change.

- **undo(): void** — restores exactly the state from before execute(), including related data and reservations, as one operation.
- **description(): String** — a short label for the history.
- **Ownership:** each Command keeps what it needs to undo itself, for example the before-state of the records it changed.

CommandHistory

- **run(c: Command): void** — executes c. Only if it succeeds is c pushed onto the undo stack, and the redo stack is cleared (a new mutation after an undo clears redo). A failed command is not recorded.
- **undo(): boolean** — undoes the most recent command and moves it to the redo stack. Returns false, changing nothing, when there is nothing to undo.
- **redo(): boolean** — re-applies the most recently undone command. Returns false when there is nothing to redo.
- **Limits:** keeps at least the 20 most recent operations. Exports and failed operations never enter history. History lives in memory only and need not survive a restart; saving after an undo stores the restored state.
- **Ownership:** CommandHistory owns the two stacks. It changes data only through Commands.

6. Implementing and testing without the UI

- Every service receives the shared CampusPlanData through its constructor. A test builds one in memory, creates the service, calls a method, and checks the returned value or the thrown exception. No console and no file is involved (NFR-04).
- Storage tests use InMemoryDataStore. JsonDataStore round trips use a JUnit temporary directory.
- Only ConsoleUI reads input or prints, so CLI work and service work can go on in parallel.
- Merge order: skeleton classes with the signatures above go into main first. Each member then implements their own service and its unit tests against that skeleton.